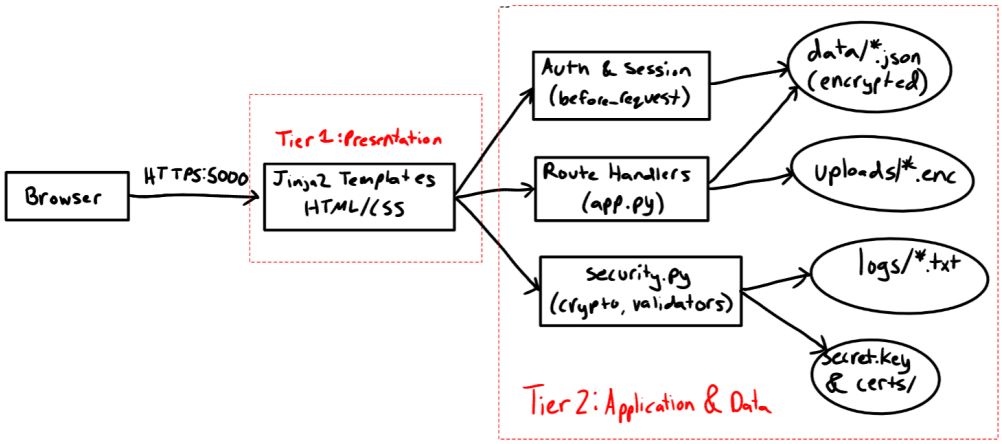
Security Design Document for SecureWebApp

Kuber Kapuriya
Computer Security CS 419

1 A. Architecture Overview

1.1 System Architecture

SecureWebApp is a 2-tier app. The presentation tier is made of Jinja2 templates rendered server side into HTML and CSS, which is what the browser actually sees. The application and data tier is the Flask process itself, which handles routing, authentication, session management, access control, encryption, and filesystem Input/Output against the local Fernet encrypted JSON stored files. Both tiers run inside the same Python process on the same host and there is no separate API service, no database server, and no reverse proxy.



The two trust boundaries that matter are the network boundary (browser to Flask, crossed only over TLS) and the process to filesystem boundary (where encryption at rest provides protection if the filesystem is read by something other than the Flask process).

1.2 Data Flow Diagrams

The two most security relevant flows are authentication and file upload. Both pass through multiple checks before any state changing action is taken.

Login flow: A POST to `/login` is rate limited per IP, checked against the account's lockout state, verified with bcrypt, and then finally turned into a session token written to the encrypted session store.

1. `POST /login` received.

2. RateLimiter check 10 requests per 60 seconds per IP, returns 429 on overflow.

3. Load and decrypt `users.json`.
4. Lockout check 5 failed attempts triggers a 15min lockout.
5. `bcrypt.checkpw` compares the submitted password against the stored hash.
6. Create session 32-byte random token written to the encrypted session store.
7. Set cookie (HttpOnly, Secure, SameSite=Strict) and redirect to `/dashboard`.

File upload flow: A POST to `/upload` passes through authentication, role gating, input validation, filename sanitization, encryption, and finally a metadata write and audit log entry.

1. `POST /upload` received.
2. `@require_auth` and `@deny_guest` decorators enforce that the caller is authenticated and not a guest.
3. `validate_file_upload` extension whitelist, MIME check, and 16 MB size cap.
4. `safe_filename` werkzeug sanitization plus a regex pass.
5. Fernet encryption (AES-128-CBC with HMAC-SHA256) of the raw file bytes.
6. Write ciphertext to `uploads/<uuid>_v<n>.enc`.
7. Update the encrypted `documents.json` with the new version record.
8. Write an audit log entry recording the upload.

Download and delete flows follow the same pattern: `@require_auth` first, then an ownership or share check (`owns_document` / `can_access_document` / `can_edit_document`), then `safe_file_path` to prevent path traversal before touching the filesystem, then the actual operation, then an audit entry.

1.3 Component Descriptions

app.py is the Flask app builder and route layer. It contains all HTTP handlers along with the before/after request hooks that enforce HTTPS redirects, session validation, and security headers. It handles app bootstrapping (creating data directories, seeding empty stores, and running a one time migration of any legacy plaintext JSON to encrypted form on first boot), TLS certificate auto generation when none is present, password hashing wrappers over bcrypt, rate limiting and account lockout on the login route, centralized cookie security flags, soft-delete and version resolution helpers used by the document routes, document ownership and share checks, secure file deletion, a per document audit view, MIME type mapping for uploads, custom error handlers for 403/404/429/500, and Jinja helpers for template rendering.

security.py is the security kernel. It contains `EncryptedStorage` (Fernet key management and encrypt/decrypt primitives), `SessionManager` (token creation, validation, expira-

tion, revocation), `SecurityLogger` (structured JSON audit events), `RateLimiter` (per IP sliding window), the input validators for usernames / emails / passwords, `safe_filename`, `safe_file_path`, `validate_file_upload`, and the authentication decorators `require_auth`, `require_role`, and `deny_guest`. Security relevant logic is here so that it can be reviewed and tested in isolation.

config.py holds constants like: session timeouts, rate limit window, file size caps, upload directory path, and the allowed file extensions and MIME types. The Flask `SECRET_KEY` is loaded from the environment or generated per process.

templates/ contains Jinja2 templates with auto escaping enabled. User controlled values are also passed through the explicit `| e` filter as a defense measure.

data/ contains the encrypted JSON stores: `users.json`, `documents.json`, `shares.json`, `sessions.json`, and `audit.json`. Even though the extension is `.json`, the contents are raw Fernet ciphertext and will not parse as JSON.

uploads/ holds the encrypted document files, named `<uuid>_v<n>.enc`. Original filenames are stored only in the encrypted metadata, not on disk.

logs/ contains `security.log` and `access.log` as plaintext JSON lines.

certs/ contains the self signed RSA-2048 TLS certificate and private key, its auto generated on first run with a 365 day validity period.

1.4 Technology Stack Justification

I chose Flask 3.0+ over Node.js/Express because Python's standard library and ecosystem give stronger support for the security features this project and also I am familiar with this ecosystem more. Also Jinja2's auto-escaping provides XSS protection by default, and Werkzeug's secure filename is the standard for upload sanitization. The setup for Flask is also less complicated in my experience. Lastly, using this Flask/Python stack lets me use PyCA which provides Fernet. Using Fernet rather than hand rolling AES avoids the most common cryptographic mistakes like IV reuse, missing authentication. The same library is used to generate the RSA-2048 self-signed certificate.

2 B. Threat Model

2.1 Asset Identification

Highest sensitivity: the Fernet master key (`secret.key`), the TLS private key (`certs/key.pem`), user password hashes (inside encrypted `users.json`), and active session tokens (inside encrypted `sessions.json`).

Confidential: uploaded document files, document metadata, share records, user email addresses, and the audit trail.

Internal: plaintext logs (which contain IP addresses, usernames, and emails) and usernames themselves.

Public: static CSS/JS files and the TLS public certificate.

2.2 Threat Enumeration

I found the following threats based on the OWASP Top 10 and the system's specific attack surface:

T1 Credential brute force or stuffing against the login endpoint.

T2 Session hijacking via cookie theft.

T3 Cross-site scripting through user controlled fields.

T4 Cross-site request forgery against state changing endpoints.

T5 Path traversal through filenames or document identifiers.

T6 Insecure direct object reference (accessing another user's document by guessing its ID)

T7 Privilege escalation with client side role manipulation.

T8 Malicious or oversized file upload.

T9 Data breach with direct filesystem access.

T10 Man in the middle interception of plaintext traffic.

T11 Denial of service against the login endpoint.

T12 File content recovery after deletion.

T13 Injection (SQL, command, or template).

T14 Session fixation leading to account takeover.

2.3 Vulnerability Assessment

The state of each threat mitigation status:

Fully mitigated: T1, T2, T3, T5, T6, T7, T8, T9, T10, T11, T12, T13
T1 is blocked by the combination of per IP rate limiting (10 requests per 60 seconds) and per account lockout (5 failures triggers a 15 minute lockout). T2 is addressed by the `HttpOnly`, `Secure`, and `SameSite=Strict` cookie flags together with 30 minute idle and 8 hour absolute session timeouts and server side revocation. T3 is blocked by Jinja2 auto escaping, the explicit `| e` filter, and a CSP with `script-src 'self'`. T5 is blocked by `safe_file_path`, which resolves paths with `abspath` and enforces base directory containment, plus `safe_filename` for uploaded names; any attempt is logged as CRITICAL. T6 is blocked by ownership and share checks on every document route. T7 is blocked because roles are stored and checked server side only, with no client supplied role data trusted. T8 is blocked by the extension whitelist, MIME check, and 16 MB size cap. T9 is mitigated by Fernet encryption of every data file and every upload. T10 is blocked by HTTPS enforcement, HSTS, and the `Secure` cookie flag. T11 is blocked by the rate limiter. T12 is mitigated by

the random and then zero overwrite routine in `_secure_delete_file`. T13 is blocked structurally since there is no SQL, no shell execution, no `eval`, and no `render_template_string` on user input.

Partially mitigated: T4, T14
T4 is strongly mitigated by `SameSite=Strict` cookies, which prevent cross origin form submissions from carrying the session cookie, but there is no explicit CSRF token and Flask-WTF is not used. T14 is mitigated because session tokens are generated server side and cannot be preset by an attacker, but there is no session regeneration on privilege change.

Could be Considered Residual: T2, T9, T12
T2 can still be a problem if TLS is removed before the request reaches the app(infrastructure problem, not app). T9 applies to log files, which are plaintext. T12 a common issue in many apps, overwritten or deleted data may still persist at the storage level.

2.4 Attack Scenarios

S1 Brute force login: An attacker scripts password guesses against a known username. The rate limiter returns HTTP 429 after 10 attempts in 60 seconds. If attempts come from rotating IPs, the per account lockout starts after 5 failures and holds for 15 minutes.

S2 Stolen session cookie: If an attacker somehow obtains a session cookie, the `Secure` flag prevents exfiltration over HTTP, `HttpOnly` prevents JavaScript access, and the 30 minute idle timeout limits the attack window. The user can log out to revoke the session server side.

S3 Insecure Direct Object Reference attempt: An attacker crafts a GET request with another user's document ID. The route handler calls `can_access_document`, which checks both ownership and the share table. A mismatch returns 403 and logs an `ACCESS_DENIED` event.

S4 Path traversal: An attacker uploads a file named `../../etc/passwd.txt`. The `werkzeug` `secure_filename` helper strips the traversal sequence, the regex in `safe_filename` rejects anything remaining that falls outside the allowed character set, and the attempt is logged at `CRITICAL`.

S5 Malicious file upload: An attacker renames `malware.exe` to `malware.pdf`. The extension whitelist accepts the `.pdf` extension but the MIME check compares against the browser reported `Content-Type`, and oversized files are rejected at the 16 MB cap.

S6 XSS via filename: An attacker uploads a file named `<script>alert(1)</script>.pdf`. `safe_filename` sanitizes the stored name, Jinja2 escapes it on display, and the CSP blocks inline script execution even if escaping failed.

S7 Guest privilege escalation: A guest user submits a POST to `/upload`. The `@deny_guest` decorator returns 403 before the handler runs and logs `ACCESS_DENIED`.

S8 Filesystem dump: An attacker gains read access to the `data/` directory. Every file is Fernet ciphertext; without `secret.key`, the contents are safe.

2.5 Risk Prioritization

Critical:

- `secret.key` compromise would decrypt all stored data. Response: OS file permissions (0600), never committed to git, `.gitignore` enforced.

High:

- Session cookie theft leading to account takeover. Response: HTTPS + `Secure` / `HttpOnly` / `SameSite=Strict` flags + short timeouts.
- CSRF on state changing actions. Response: `SameSite=Strict`.

Medium:

- Plaintext logs with PII. Response: could do log encryption with fernet.

Low:

- Storage data persistence after secure delete. Response: can do full disk encryption at the OS.

3 C. Security Controls

3.1 Authentication

Password hashing: Passwords are hashed with bcrypt at cost factor 12. Salts are generated automatically and embedded in the hash string. Implementation lives in `hash_password` and `check_password` in `security.py`. Tested in `test_auth.py` by verifying that wrong passwords are rejected, that stored values are hashes rather than plaintext. The known limitation is that bcrypt silently truncates inputs at 72 bytes so very long passphrases would all collapse to the same hash prefix. The mitigation is to enforce a maximum length in `validate_password`, which is currently unbounded on the upper end.

Password complexity: Passwords must be at least 12 characters and include uppercase, lowercase, a digit, and a special character. Enforced in `validate_password` and tested in `test_input_validation.py`. The known limitation is that there is no check against common password dictionaries so integrating some relavent API is a future option.

Account lockout: Five failed login attempts trigger a 15 minute lockout, tracked via `failed_attempts` and `locked_until` fields on the user record. The login route checks `locked_until` before attempting bcrypt comparison, which also prevents timing based user enumeration during lockout. Tested in `test_auth.py`. The lockout is persistent across server restarts. The limitation is that there is no admin UI to manually unlock a user, so unlocking currently requires editing data file directly.

Rate limiting: The login endpoint is limited to 10 attempts per IP per 60 seconds with an in memory sliding window in `RateLimiter`. Exceeding the limit returns HTTP 429. Tested in `test_auth.py::test_login_rate_limit`. Limitations: state resets on server restart, the

limiter is single process only, and IP based limiting is spoofable behind a proxy that passes through `X-Forwarded-For`. A persistent store like Redis would be needed for mitigation in production.

3.2 Session Management

Session tokens: Generated with `secrets.token_urlsafe(32)`, giving 256 bits of entropy. Created in `SessionManager.create_session`. Tested in `test_session.py` for creation, validation, destruction, expiration, and schema correctness. One token per user is supported. No multiple device session list is exposed.

Session timeout: 30 minute idle timeout and 8 hour absolute lifetime, enforced in `validate_session` by comparing `last_activity` and `created_at` against the current time. Tested by fast forwarding timestamps in the test fixtures. The limitation is that timeouts are only reevaluated on the next request. A stolen cookie remains valid until its window closes.

Cookie security: Session cookies are set with `HttpOnly`, `Secure`, `SameSite=Strict`, and `max_age=1800`. Configured with `_cookie_kwargs` in `app.py`. In testing setups, cookies are being configured with flags to make sure they behave correctly during integration tests. The `Secure` flag is relaxed in `TESTING` mode so that tests can run against HTTP. In actual production, this testing mode should be turned off.

Server side revocation: Deleting a token from `sessions.json` invalidates the cookie on the next request. `SessionManager.destroy_session` runs on logout, and `cleanup_expired` runs on every request. Tested in `test_session.py`. The limitation is performance, since the cleanup touches the encrypted session file on every request. A background cleanup task would scale better.

3.3 Authorization

Role based access control: Three roles exist: admin, user, and guest. They are enforced by the `require_role(*roles)` and `deny_guest` decorators. Roles are stored server side in `users.json`. No client supplied role is ever trusted. Tested in `test_access_control.py`. The limitation is that there is no admin UI for role changes, so they currently need direct data edits.

Document ownership and share checks: Every document route checks one of `owns_document`, `can_access_document`, or `can_edit_document`. These helpers live in `app.py` and admin bypass is applied at the view and download routes only. Tested with non owner 403s and share role differentiation in `test_access_control.py`. The limitation is that share lookups iterate all shares on every request (O(n)). Indexing by user ID or moving to a database would fix this.

Editor role constraints: Users with the editor role on a shared document can download and upload new versions, but cannot delete the document or reshare it. These checks live alongside the ownership helpers and are verified in the access control tests.

Guest share downgrade: When a document is shared with a user whose role is guest,

the requested share role is coerced to `viewer` regardless of what the sharing form submitted. Tested in `test_access_control.py::test_guest_share_as_editor_saved_as_viewer`. The limitation is that existing `editor` shares to a user who is later demoted to guest are not downgraded. The mitigation is to recheck the target user's role inside `can_edit_document` rather than trusting only the stored share role.

3.4 Input Validation

Username, email, and password validation: Regex based allowlists in `validate_username`, `validate_email`, and `validate_password` reject any input that does not match on the first failure. Tested against SQL injection, XSS payloads, and length boundary cases in `test_input_validation.py`. The email regex is not RFC 5321-complete, but it accepts the common cases and rejects obviously invalid input which is enough.

File upload validation: Uploads must match the extension whitelist (`pdf`, `txt`, `docx`, `png`, `jpg`, `jpeg`), the MIME type allowlist, and the 16 MB size cap. Filenames pass through `safe_filename`, which layers `werkzeug.secure_filename` with an additional regex pass and an extension check. Tested with an `.exe` rejection case, an oversized file case, and an XSS in filename case. The key limitation is that the MIME check uses the browser reported `Content-Type`, which a client can spoof. Server side MIME detection via `python-magic` is the mitigation.

Output encoding: All user controlled data is HTML escaped before rendering, both by Jinja2's default autoescape and by an explicit `| e` filter. Attempts at XSS by using usernames or filenames are verified not to reflect in `test_input_validation.py`. The Content-Security-Policy currently includes `'unsafe-inline'` in `style-src`, which is needed because a few inline style attributes remain in templates. Extracting these to the external stylesheet would let the current relaxed encoding be removed.

3.5 Transport Security

TLS enforcement: All routes run over HTTPS, and plain HTTP is redirected with a 301 by the `before_request` hook. The server runs with `ssl_context` pointing at the self signed certificate. HSTS is sent with `max-age=31536000` so that browsers remember the redirect. The known limitation is the browser warning produced by the self signed certificate. A CA signed certificate would be required for any public deployment of the app.

Security headers: The `after_request` hook sets Content-Security-Policy, X-Frame-Options (`DENY`), X-Content-Type-Options (`nosniff`), X-XSS-Protection, Referrer-Policy (`no-referrer`), Permissions-Policy, and HSTS. Header presence is checked in integration tests.

3.6 Audit Logging

Security event logging: `SecurityLogger.log_event` is called at every security relevant boundary like: authentication successes and failures, file operations, access denials, and path traversal attempts. Entries are JSON structured with severity levels and event type constants. Tested in `test_audit.py` for checking schema, event coverage, and filtering. The

known limitations are that the logs are plaintext on disk and contain PII (IP addresses, usernames, emails) and there is no log rotation. I kept the logs unencrypted so that I can read them but I could encrypt them in filesystem and have them be viewable in admin dashboard in app. Encrypting the log files and redacting email addresses are the potential mitigations.

4 D. Data Protection

4.1 Data Classification

Secret data would break the entire system if exposed: `secret.key`, `certs/key.pem`, the bcrypt password hashes stored inside encrypted `users.json`, and active session tokens inside encrypted `sessions.json`.

Confidential data is the protected content the system serves: uploaded document bytes (`.enc` files), document metadata in `documents.json` (filenames, owner IDs, version history), share records in `shares.json`, user emails in `users.json`, and the audit trail in `audit.json`.

Internal data is application data that is not user content but still reveals information about activity: the plaintext logs, which contain IPs and identifiers, and raw usernames.

Public data needs no confidentiality: the static CSS and JS files, and the TLS certificate.

4.2 Encryption Methods

Document files at rest: Encrypted with Fernet (AES-128-CBC with HMAC-SHA256 and a timestamp) with `cryptography.fernet`. The 256-bit Fernet key is split into a 128-bit AES encryption key and a 128-bit HMAC signing key. The IV is generated randomly per encryption call by the library. Authentication is built in so any tampering with the ciphertext produces a decryption failure rather than corrupted plaintext. Files are at `uploads/` folder.

JSON data stores at rest: The same Fernet key is used to encrypt the entire contents of `users.json`, `documents.json`, `shares.json`, `sessions.json`, and `audit.json`. On disk these files contain raw Fernet ciphertext. Though they have the `.json` extension, they will not parse as JSON without first being decrypted.

Data in transit: TLS is served by Werkzeug using the generated `ssl_context`. The certificate is RSA-2048 with SHA-256 signing, auto generated on first run and valid for 1 year. All HTTP is redirected to HTTPS. HSTS is set for one year with `includeSubDomains`.

Passwords: Passwords are not encrypted but hashed with bcrypt at cost 12. The hash format `<salt><hash>` embeds the salt, so no separate salt storage is required.

4.3 Key Management

Fernet master key: This is the most sensitive asset in the system since anyone who reads it can decrypt every document and every data store. It must be kept out of version control (gitignore), kept off any shared folder, and kept readable only by the OS account that runs the application. Backup is the operator's responsibility since loss of the key means permanent loss of all encrypted data, and any backup copy must be protected to the same standard as

the original (encrypted storage, offline media, or a password manager's secure notes).

TLS private key: The private key in `certs/key.pem` must also be excluded from version control and restricted to the app's OS account. Because the certificate is self signed and auto regenerated when the `certs/` directory is deleted, rotation is easy. If the key is ever suspected of compromise, delete the directory and restart.

Flask `SECRET_KEY`: Since the app doesn't rely on Flask's built in session, this key is lower sensitivity. The environment variable itself should never be committed to the repo or checked into shell history files.

4.4 Secure Deletion Procedures

Document deletion is implemented in `_secure_delete_file` in `app.py` and triggered by `POST /document/<id>/delete`. The owner check runs first, then for each version of the document these steps are followed:

1. Read the file size.
2. Open the file in read and write mode.
3. Overwrite the entire file with `os.urandom(size)` (cryptographically random bytes).
4. `fsync` to flush the random bytes to physical media.
5. Seek back to offset 0.
6. Overwrite the entire file with zero bytes.
7. `fsync` again to flush the zeros.
8. Unlink the file (remove the directory entry).
9. On any exception, log at `WARNING` severity and reraise the error.

When an owner deletes a document, every version, current and old, are wiped together. Old versions are retained in encrypted form on disk while the document is active (they are not viewable or downloadable through the UI, only tracked in metadata), so a full delete goes through the entire version list and securely overwrites each `.enc` file in turn. After every version is wiped, the document's metadata record is updated with `is_deleted=True` rather than being removed, and an audit entry is written with the count of versions wiped. Metadata (original name, timestamps, version list), the audit entries that reference the document, and the share records that reference it are all preserved intentionally, so that admin can still see. There is no perversion delete action so securely removing a single old version would require deleting the whole document.

Limitations: On SSDs, data may not be fully overwritten because of how storage is managed, so old copies can still exist until they are cleaned up later. Some file systems also keep older versions of files through snapshots or journals. However, even if this leftover data exists, it is encrypted with Fernet, so it cannot be read without the encryption key. This means encryption is the main protection, while secure deletion is only an extra step. For

stronger protection, full disk encryption (like BitLocker) should be used so any leftover data is still unreadable.